NANYANG TECHNOLOGICAL UNIVERSITY

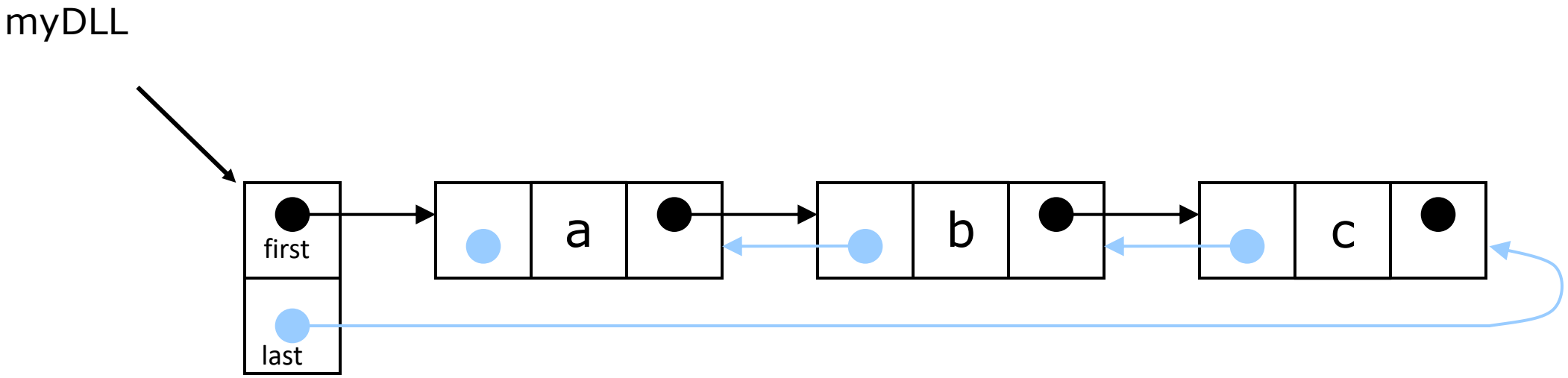
Data Structures & Algorithms in Python: Doubly & Circular LinkedLists

Dr. Owen Noel Newton Fernando

College of Computing and Data Science (CCDS)

Doubly-linked lists

Here is a doubly-linked list (DLL):



- Each Link contains a datum, a reference to its successor (if any), and a reference to its predecessor (if any)
- The header points to the first Link in the list *and* to the last Link in the list (or contains None's if the list is empty)

Content Copyright Nanyang Technological University

A DLL's Link

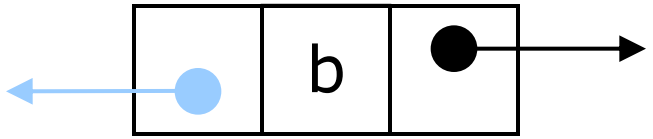
```
class Link(LinkedList.Link):        # One datum in a linked list
   def __init__(self, datum,        # Constructor with datum
                next=None,          # and optional next and
                previous=None):     # previous pointers
      self.__data = datum
      self.__next = next            # reference to next item in list
      self.__previous = previous    # reference to previous item

   def getData(self): return self.__data   # Accessors
   def getNext(self): return self.__next
   def getPrevious(self): return self.__previous
   def setData(self, d): self.__data = d
   def setNext(self, link):              # Accessor that enforces type
      if link is None or isinstance(link, Link):
         self.__next = link
      else:
         raise Exception("Next link must be Link or None")
   def setPrevious(self, link):          # Accessor that enforces type
      if link is None or isinstance(link, Link):
         self.__previous = link
      else:
         raise Exception("Previous link must be Link or None")
```

Content Copyright Nanyang Technological University

A DLL's Link (code Explanation)

```
def __init__(self, datum,        # Constructor with datum
             next=None,          # and optional next and
             previous=None):     # previous pointers
    self.__data = datum
    self.__next = next           # reference to next item in list
    self.__previous = previous   # reference to previous item
```

This is the constructor for the **Link** class. It initializes a Link object with data and optional references to the next and previous nodes.



Parameters:

datum: The data (or value) to store in this node.
next: Reference to the next node in the list (default is None).
previous: Reference to the previous node in the list (default is None).
Function: Sets the node's data and links to the next and previous nodes.

Content Copyright Nanyang Technological University

A DLL's Link (code Explanation)

```
def getData(self): return self.__data   # Accessors
def getNext(self): return self.__next
def getPrevious(self): return self.__previous
```

getData():
Returns the data stored in the node.
Return Value: The data held in self.__data.

getNext Method():

Returns the reference to the next node.
Return Value: The next node in the list (self.__next) or None if there's no next node

getPrevious Method():

Returns the reference to the previous node.
Return Value: The previous node in the list (self.__previous) or None if there's no previous node.

Content Copyright Nanyang Technological University

A DLL's Link (code Explanation)

```
def setData(self, d): self.__data = d
def setNext(self, link):              # Accessor that enforces type
    if link is None or isinstance(link, Link):
        self.__next = link
    else:
        raise Exception("Next link must be Link or None")
```

setData():
Updates the data stored in the node.
Parameters: **d** - The new data to set in the node.

setNext():
Updates the reference to the next node.
Parameters: **link -** The new Link object to set as the next node, or None.
Function:
Checks if link is either None or an instance of Link. If valid, updates self.__next with link.If invalid, raises an exception with an error message.

Content Copyright Nanyang Technological University

A DLL's Link (code Explanation)

```
def setPrevious(self, link):          # Accessor that enforces type
    if link is None or isinstance(link, Link):
        self.__previous = link
    else:
        raise Exception("Previous link must be Link or None")
```

setPrevious():

Updates the reference to the previous node.
Parameters: **link -** The new Link object to set as the previous node, or None.
Function:
Checks if link is either None or an instance of Link. If valid, updates self.__previous with link. If invalid, raises an exception with an error message.

Content Copyright Nanyang Technological University

The DLL class

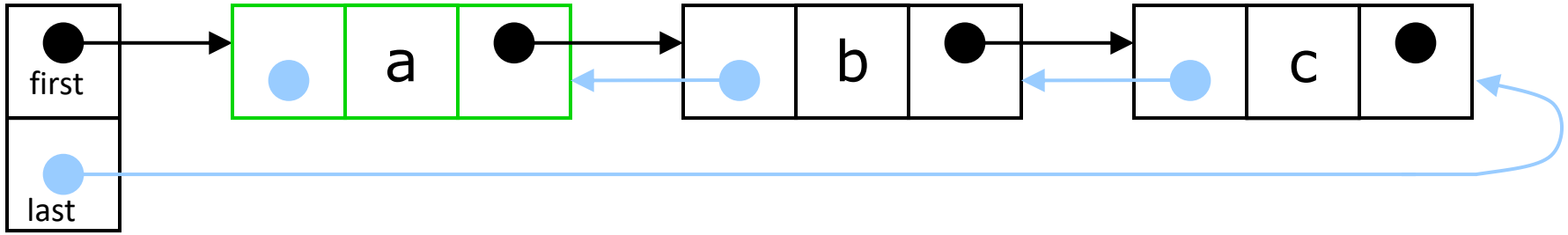
```
class DoublyLinkedList(LinkedList.LinkedList):
   def __init__(self):                    # Constructor
      self.__first, self.__last = None, None

   def getFirst(self): return self.__first # Accessors
   def getLast(self): return self.__last

   def setFirst(self, link):              # Set first link
      if link is None or isinstance(link, Link): # Check type
         self.__first = link
         if (self.__last is None or       # If list was empty or
             link is None):               # list is being truncated
            self.__last = link            # update both ends
      else: raise Exception("First link must be Link or None")

   def setLast(self, link):               # Set last link
      if link is None or isinstance(link, Link): # Check type
         self.__last = link
         if (self.__first is None or      # If list was empty or
             link is None):               # list is being truncated
            self.__first = link           # update both ends
      else: raise Exception("Last link must be Link or None")
```

Content Copyright Nanyang Technological University

DLL class cont. - Inserting at either end

```
def insertFirst(self, datum): # Insert a new datum at start of list
   link = Link(datum,         # New link has datum
               next=self.getFirst()) # and precedes current first
   if self.isEmpty():         # If list is empty,
      self.setLast(link)      # insert link as last (and first)
   else:                      # Otherwise, first Link in list
      self.getFirst().setPrevious(link) # now has new Link before
      self.setFirst(link)     # Update first link

insert = insertFirst          # Override parent class insert()
```

The **insertFirst method** inserts a new node (or link) at the beginning of the doubly linked list.

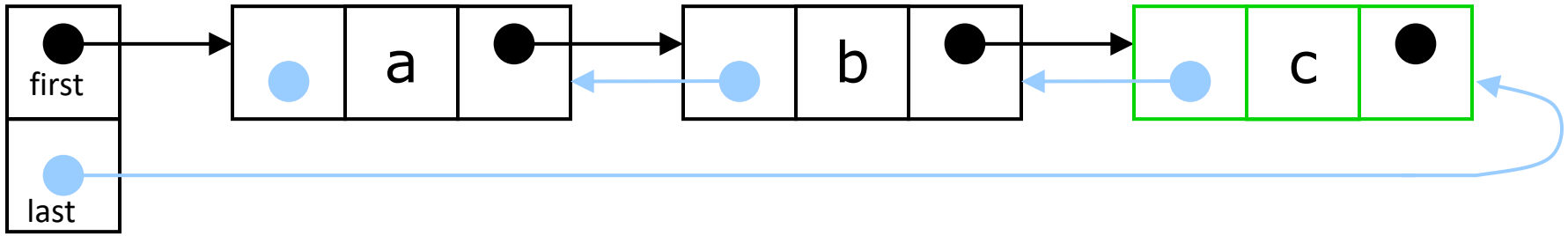


Content Copyright Nanyang Technological University

DLL class cont. - Inserting at either end

```
def insertLast(self, datum):  # Insert a new datum at end of list
   link = Link(datum,         # New link has datum
               previous=self.getLast()) # and follows current last
   if self.isEmpty():         # If list is empty,
      self.setFirst(link)     # insert link as first (and last)
   else:                      # Otherwise, last Link in list
      self.getLast().setNext(link) # now has new Link after
      self.setLast(link)      # Update last link
```

The **insertLast method** inserts a new node (or link) at the end of the doubly linked list.



Content Copyright Nanyang Technological University

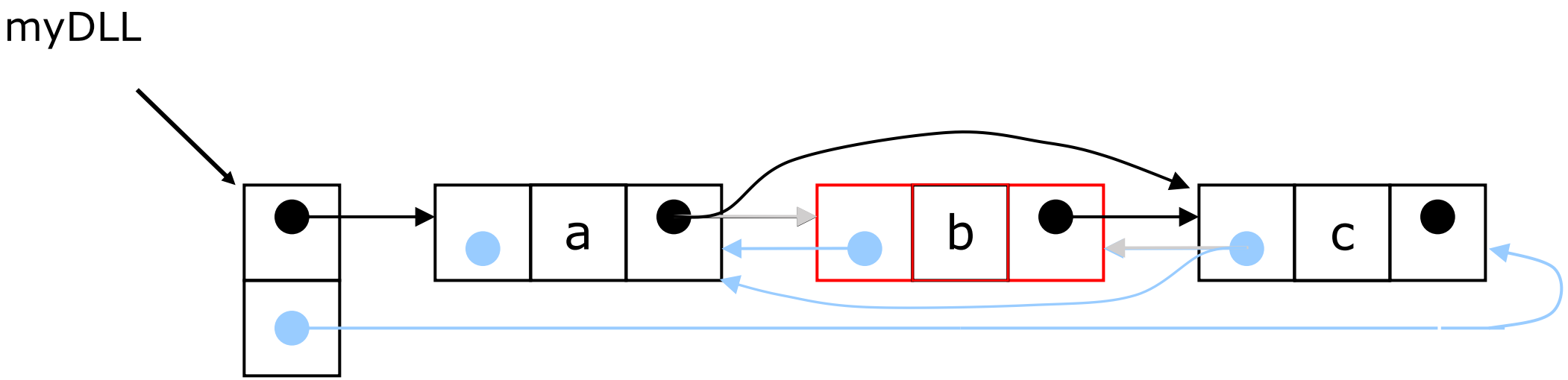
Delete a Link from DLL

```
def delete(self, goal,          # Delete the first Link from the
           key=identity):       # list whose key matches the goal
   link = self.find(goal, key)  # Find matching Link object
   if link is None:             # If not found, raise exception
      raise Exception("Cannot find link to delete in list")
   if link.isLast():            # If matching Link is last,
      return self.deleteLast()  # then delete from end
   elif link.isFirst():         # If matching Link is first,
      return self.deleteFirst() # then delete from front
   else:                        # Otherwise it's a middle link
      link.getNext().setPrevious( # Set next link's previous
         link.getPrevious())    # to link preceding the match
      link.getPrevious().setNext( # Set previous link's next
         link.getNext())        # to link following the match
      return link.getData()     # Return deleted data item
```

Content Copyright Nanyang Technological University

Deleting a Link from a DLL

- Link insertion or deletion from a DLL involves changing *two* links



- Deletion of the first Link or the last Link is a special case
- Garbage collection will take care of deleted Link objects

Content Copyright Nanyang Technological University

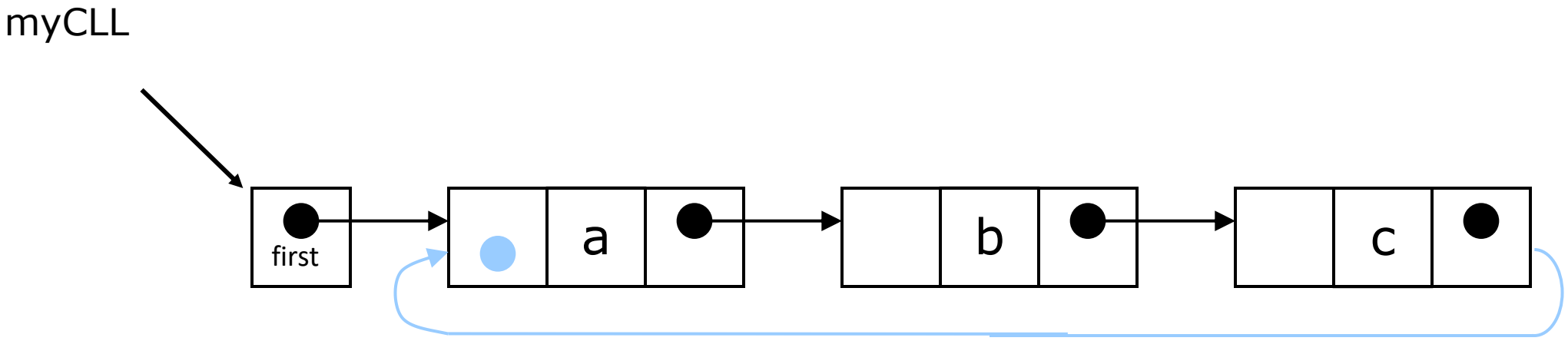
DLLs compared to SLLs

- **Advantages:**
  - Can be traversed in **either direction** (may be essential for some programs)
  - Some operations, such as deletion and inserting before a link, become easier
- **Disadvantages:**
  - Requires **more space**
  - List manipulations are **slower** (because more links must be changed)
  - Greater chance of having bugs (because more links must be manipulated)

Content Copyright Nanyang Technological University

Circular Linked lists

Here is a Circular-linked list based on a Singly Linked List(CLL):



- Each Link contains a datum, a reference to its successor (if any)
- The header points to the first Link in the list, *and* the last Link in the list also points to the first Link forming a **continuous loop.**
- Circular Linked Lists can be formed using either a **Singly Linked List** or a **Doubly Linked List.**

Content Copyright Nanyang Technological University

CLLs compared to SLLs

- **Advantages:**
  - **Constant Traversal**: Traversal from any node will eventually return to the starting node, making circular lists ideal for applications needing repeated cycling.
- **Disadvantages:**
  - **Risk of Infinite Loops**: Errors in traversal logic can easily lead to infinite loops due to the circular structure, especially if stop conditions are missed.

Content Copyright Nanyang Technological University

Practical Applications - DLL

Doubly linked lists (DLLs) are useful in applications where **bidirectional traversal**, **flexible insertion**, and **deletion** are essential. Here are some common real-world applications of DLLs:

Browser History Navigation:

Browsers use doubly linked lists to manage page navigation history. Each page visit is a node, allowing users to move back and forth between pages seamlessly.

Undo and Redo Operations:

In text editors and applications with undo/redo functionality, DLLs store each state change, allowing users to go forward or backward in the edit history efficiently.

Implementation of LRU Cache (Least Recently Used):

In memory management, doubly linked lists are used to track recently accessed items. Nodes store cache entries, allowing the system to efficiently add new entries and remove the least recently accessed ones, with both directions accessible.

Content Copyright Nanyang Technological University

Practical Applications - CLL

Practical Applications of Circular Linked Lists:

Round-Robin Scheduling:

Operating systems use circular linked lists for round-robin scheduling to cycle through processes, moving to the next in a loop, which ensures each task gets its turn.

Circular Buffers:

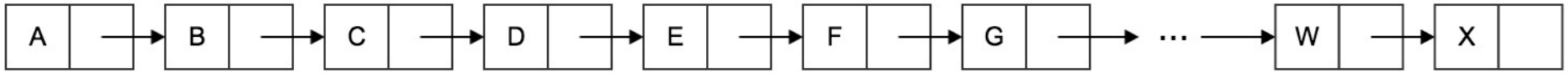
Circular linked lists are used in circular buffers, which are common in streaming, data buffering, and telecommunications, allowing efficient handling of incoming and outgoing data.

Content Copyright Nanyang Technological University

Cool trick – finding the midpoint of a linked list (single, double, or circular)

making only one pass over the nodes

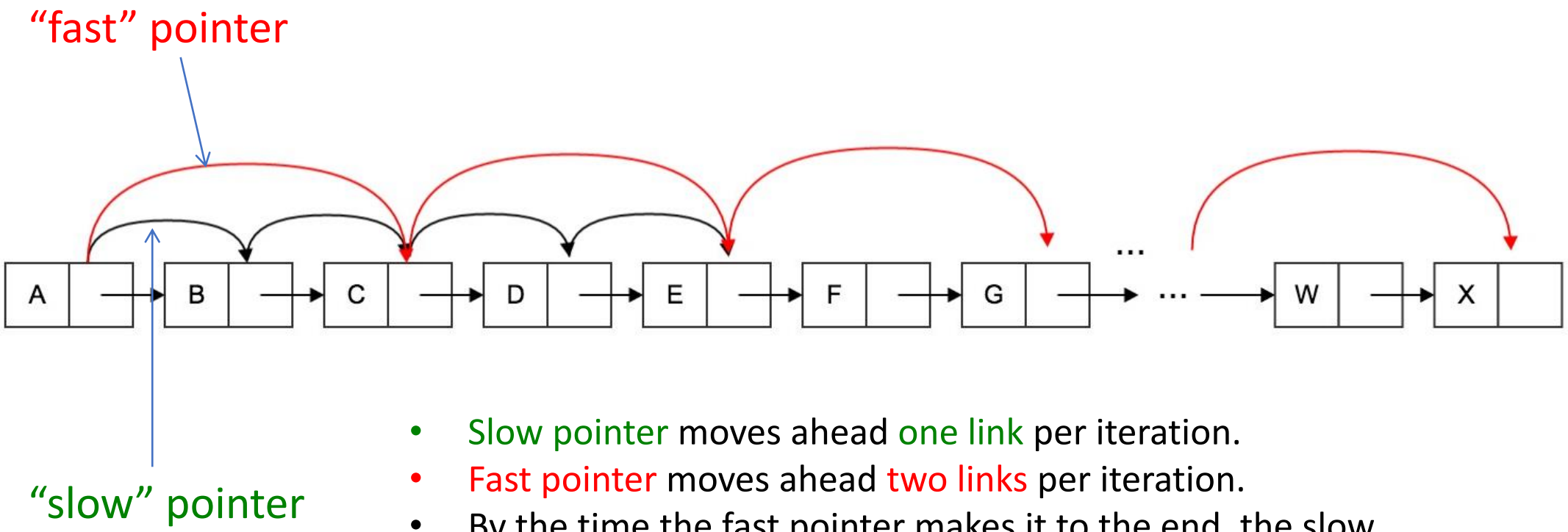
Want to find the midpoint of a linked list?



Content Copyright Nanyang Technological University

Cool trick – finding the midpoint of a linked list

making only one pass over the nodes

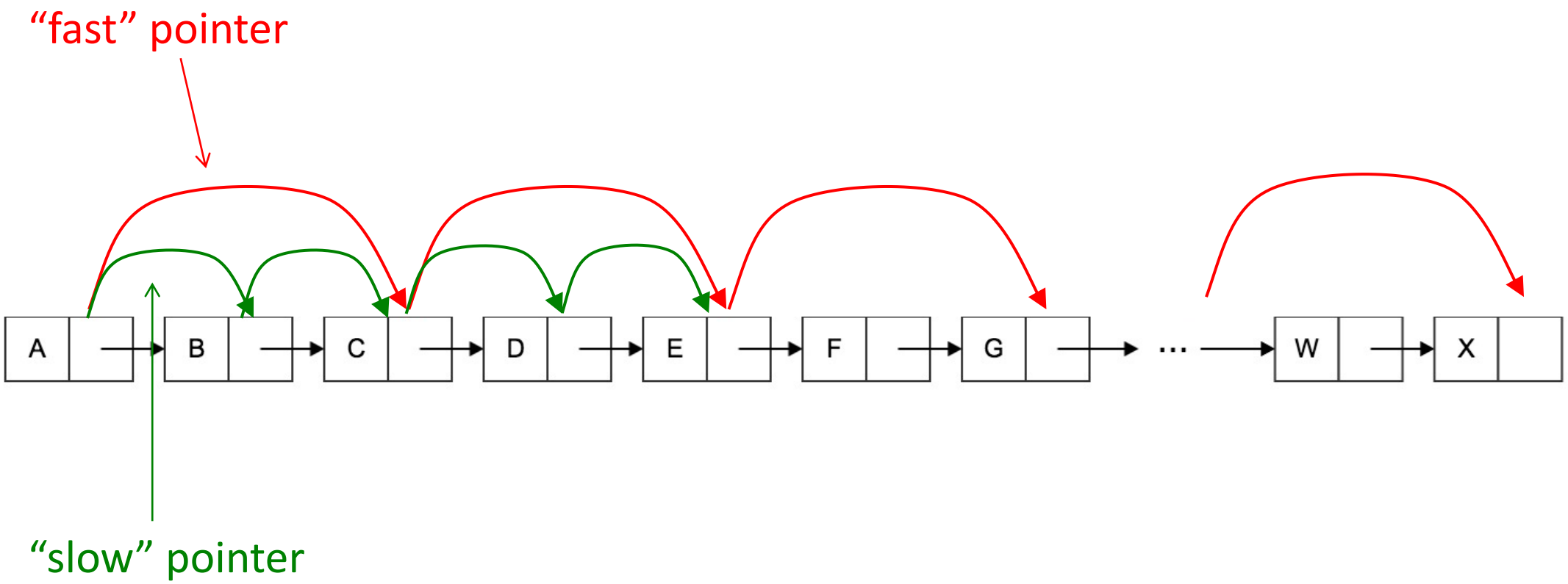


- Slow pointer moves ahead one link per iteration.
- Fast pointer moves ahead two links per iteration.
- By the time the fast pointer makes it to the end, the slow pointer will be at the midpoint.

Content Copyright Nanyang Technological University

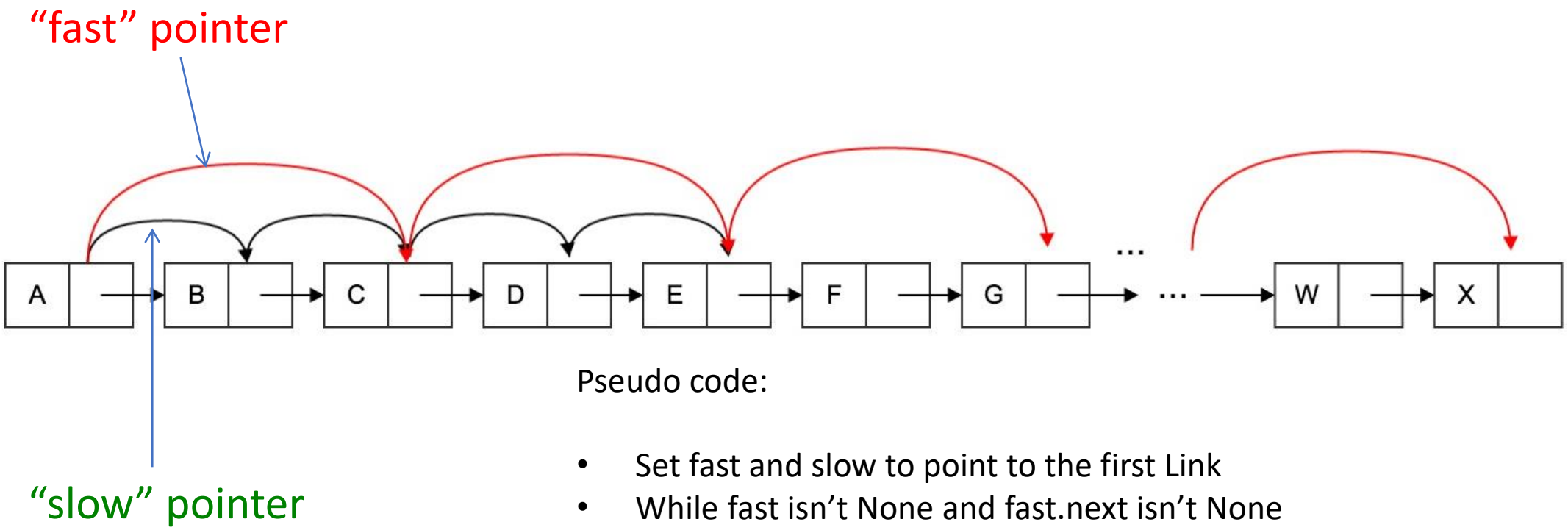
Cool trick – finding the midpoint of a linked list

making only one pass over the nodes



Content Copyright Nanyang Technological University

Cool trick – finding the midpoint of a linked list
continued, with pseudo code



Pseudo code:

- Set fast and slow to point to the first Link
- While fast isn't None and fast.next isn't None
- set slow to slow.next
- set fast to fast.next.next
- slow is now at the end of the first half of the list

Content Copyright Nanyang Technological University